

RELATÓRIO DE AUDITORIA DE SEGURANÇA

Chronos System

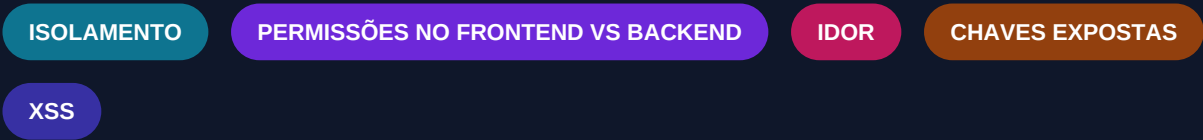
Isolamento · Permissões Frontend vs Backend · IDOR
Chaves Expostas · XSS

Repositório: c1c3ru/ChronosSystem · Branch: main
Data da auditoria: 2026-08-29 · Auditor: Claude Code — skill audit-security

ACHADOS POR SEVERIDADE



CATEGORIAS AUDITADAS



7 achados e 7 pontos fortes verificados linha a linha no código-fonte atual.

1. Metodologia e Reconhecimento de Stack

Antes de procurar qualquer vulnerabilidade, o stack tecnológico do repositório foi mapeado a partir dos arquivos de configuração reais (package.json, prisma/schema.prisma, next.config.mjs, .env.example) — não assumido a partir de documentação. A tabela abaixo resume o que foi encontrado:

Camada	Tecnologia
Framework	Next.js 16.2.6 (App Router, monolito full-stack)
Linguagem	TypeScript 5.x + React 18.2
ORM / Banco	Prisma 5.22 + PostgreSQL (sem Supabase — DATABASE_URL/DIRECT_URL diretos)
Autenticação	NextAuth.js v4 (JWT strategy) — Credentials (bcrypt) + Google OAuth com allowlist de domínio institucional
Autorização de rotas	Next.js Middleware — arquivo proxy.ts (convenção renomeada de middleware.ts no Next 16)
Cache / Rate limit	Redis (ioredis) com fallback em memória
Terminais físicos	Kiosk web (PWA) com QR HMAC-SHA256 + TOTP client-side
Geração de PDF	pdfmake (client) — @react-pdf/renderer NÃO está mais instalado
Deploy	Vercel (principal) + suporte a self-host via PM2/ecosystem.config.cjs e cron via GitHub Actions

Com o stack mapeado, cada uma das 5 categorias solicitadas foi reinterpretada especificamente para este projeto (monolito Next.js App Router, sem Supabase, com terminais físicos de kiosk), em vez de aplicar uma definição genérica:

Isolamento

Neste stack (monolito Next.js single-tenant + terminais físicos de kiosk públicos), Isolamento significa: (1) o segredo mestre QR_SECRET nunca deve alcançar um dispositivo cliente; (2) o comprometimento de um terminal físico não deve comprometer os demais; (3) endpoints intencionalmente públicos (/api/kiosk/*) não devem vaziar mais dado do que o estritamente necessário para a tela de quiosque.

Permissões no Frontend vs Backend

Como o Next.js renderiza páginas React no servidor mas o middleware de autorização roda em um arquivo separado (proxy.ts) do resto da lógica de rota, a pergunta central é: toda decisão de autorização visível na UI (esconder um botão, ocultar um menu) tem um espelho obrigatório no servidor (middleware + handler de API), ou existe algum caminho que confia apenas no que o cliente decidiu não mostrar?

IDOR

Como quase todo recurso de negócio (registros de ponto, justificativas, saldo de horas, contratos) é modelado em Prisma com um campo userId e exposto via rotas dinâmicas (/api/recurso/[id]) ou parâmetros de query (?userId=), IDOR aqui significa: buscar/alterar o registro de OUTRO usuário trocando o id/userId na URL, sem que o handler confira session.user.id ou o role do chamador.

Chaves Expostas

Além do sentido clássico (.env commitado, chave hardcoded em string), neste projeto Next.js App Router qualquer módulo importado por um Client Component ('use client') é enviado ao bundle JS do navegador — logo, uma constante 'secreta' em lib/ que seja importada por uma página client-side está, por definição, exposta

a qualquer visitante, mesmo sem nunca aparecer em uma resposta de API.

XSS

Como a superfície principal é React/JSX (que escapa automaticamente), XSS aqui foi buscado especificamente nos pontos onde o código sai desse trilho seguro: `dangerouslySetInnerHTML`, `innerHTML`, e — o padrão mais relevante encontrado neste projeto — geração de HTML via concatenação de string pura fora do JSX (respostas de API que devolvem `text/html` construído manualmente).

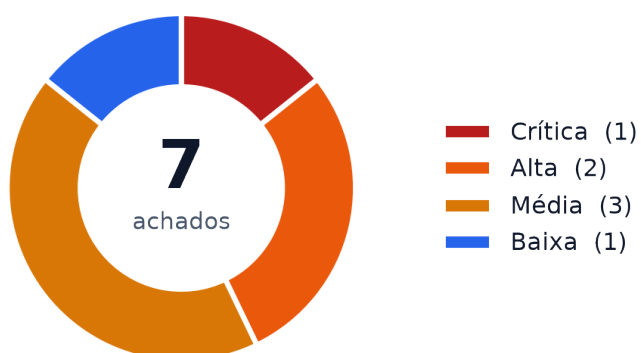
Nota metodológica

`docs/SECURITY.md` descreve uma arquitetura NestJS (`class-validator`, `ThrottlerModule`, `Helmet`, `JWT` em `localStorage`) que não corresponde ao código real (`Next.js/NextAuth/Prisma`). Trata-se de documentação desatualizada/genérica — não foi usada como fonte de verdade nesta auditoria; todo achado aqui foi verificado diretamente no código-fonte atual.

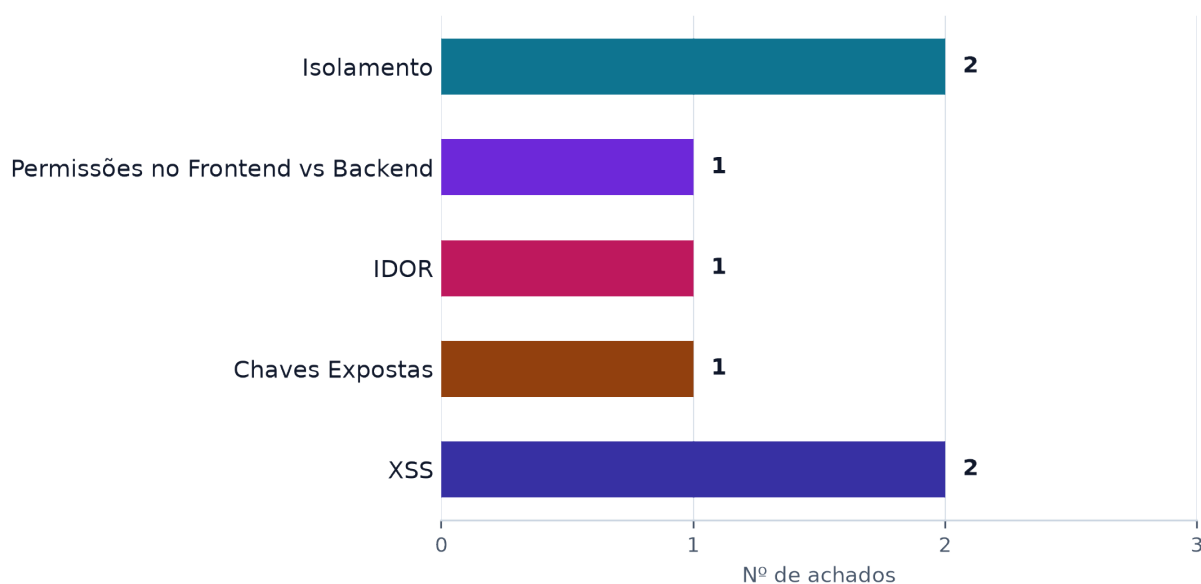
2. Resumo Executivo

A varredura cobriu o repositório inteiro e resultou em **7 achados** nas 5 categorias solicitadas, além de **7 pontos fortes** verificados diretamente no código (não apenas assumidos a partir de documentação). O achado mais severo (**F1, crítico**) permite que qualquer pessoa capaz de criar uma conta se auto-promova a administrador do sistema.

Distribuição por severidade



Achados por categoria



3. Pontos Fortes Verificados

Práticas de segurança corretas identificadas e confirmadas linha a linha no código-fonte atual (não em documentação, que pode estar desatualizada).

ID	Categoria	Arquivo:Linha	Descrição
S1	Isolamento	app/api/kiosk/init/route.ts:50-56	Segredo por-máquina derivado via HMAC, nunca o QR_SECRET mestre. machineSecret = HMAC-SHA256(QR_SECRET, machineId) é calculado no servidor e devolvido ao terminal físico; o QR_SECRET em si nunca sai do backend. Um terminal comprometido vazava apenas o segredo daquela máquina, não o segredo mestre do sistema.
S2	Isolamento	lib/qr-security.ts:98-103, 170-175, 262-265	Comparações timing-safe consistentes em toda a lógica de criptografia. validateSecureQR, validateClientSecureQR e verifyRecordHash usam crypto.timingSafeEqual com checagem de tamanho antes de comparar, prevenindo ataques de timing contra assinaturas HMAC e a cadeia de hash de auditoria. Mesmo padrão replicado em app/api/kiosk/init/route.ts:16-19 para o segredo de provisionamento.
S3	Permissões no Frontend vs Backend	proxy.ts:96-134	Gate de autorização por role roda no servidor, independente da UI. isAdminOnlyPath/isEmployeeOnlyPath/isAdminApiPath (lib/auth-policy.ts:29-61) são checados no Next.js Middleware antes de qualquer página ou rota de API executar. Verificado contra o código-fonte compilado do Next.js instalado (node_modules/next/dist/lib/constants.js:289 e dist/build/index.js:614-651): 'proxy.ts' é a convenção correta e intencional do Next 16 para o antigo middleware.ts — não é um arquivo morto.
S4	Permissões no Frontend vs Backend	lib/auth-helpers.ts:24-122	Helpers de autorização centralizados e reutilizáveis. requireAuth, requireRole, requireAdmin e requireUserAccess encapsulam a lógica de autorização em vez de cada rota reimplementar a checagem manualmente — reduz a chance de um handler futuro esquecer uma verificação.
S5	IDOR	app/api/attendance/[id]/route.ts +4 rotas:56-61 (e hour-balance:40, users/contract:65, attendance:41-54, employee/justifications/[id]:21-23)	Checagem de posse consistente nos endpoints de dados por-usuário. Todos os endpoints auditados que aceitam um id ou userId vindo do cliente conferem explicitamente session.userId === alvo OU role ADMIN/SUPERVISOR antes de responder. employee/justifications/[id] usa o padrão correto findFirst({ where: { id, userId: session.userId } }) em vez de findUnique({ where: { id } }), a forma certa de evitar IDOR no Prisma.
S6	Chaves Expostas	.gitignore:17-26; .github/workflows/*.yml:17-26	Higiene de segredos correta no repositório e no CI. Nenhum arquivo .env real está rastreado pelo git (confirmado via 'git ls-files' — apenas .env.example aparece). Os workflows do GitHub Actions referenciam segredos exclusivamente via \${ secrets.X } criptografado, sem nenhum echo ou log de valor em texto plano.

ID	Categoria	Arquivo:Linha	Descrição
S7	XSS	app/, components/, lib/ (varredura completa):N/A – ausência confirmada por grep em todo o repositório	Zero uso de dangerouslySetInnerHTML, innerHTML ou eval no código first-party. Busca em todo o código-fonte (excluindo node_modules/.next) não encontrou nenhuma ocorrência de dangerouslySetInnerHTML, atribuição a .innerHTML, eval() ou new Function(). Todo dado de usuário renderizado via JSX se beneficia do escape automático do React — a única exceção real ao 'trilho seguro' do JSX é o Achado F3.

4. Achados Detalhados

Cada achado foi localizado e verificado diretamente no código-fonte (arquivo e linha citados), com severidade justificada pelo impacto e pela facilidade de exploração.

F1 — Lista de matrículas SIAPE "administradoras" hardcoded e enviada ao bundle do navegador permite auto-promoção a ADMIN

CRÍTICA

Categoria: Chaves Expostas (relacionado: Permissões no Frontend vs Backend)

Local: lib/admin-siape.ts:8-57; app/api/auth/complete-profile/route.ts:71, 144; app/auth/complete-profile/page.tsx:1, 21, 677-688; app/admin/users/new/page.tsx:1, 30, 611-616

Descrição: lib/admin-siape.ts exporta um array hardcoded com 47 matrículas SIAPE (incluindo nome completo e matrícula real do proprietário do sistema, em comentário na linha 10). determineRoleFromSiape() apenas confere se o número está nesse array — não há nenhuma verificação de identidade real contra RH ou outra fonte. app/api/auth/complete-profile/route.ts chama essa função diretamente sobre o campo siapeNumber vindo do corpo da requisição (não confiável) e grava o resultado em User.role (linha 144) sem nenhuma outra validação de autorização. Como lib/admin-siape.ts também é importado por dois Client Components ('use client' confirmado em app/auth/complete-profile/page.tsx:1 e app/admin/users/new/page.tsx:1), o Next.js inclui o array inteiro no bundle JavaScript enviado a qualquer visitante — e a própria tela de completar perfil exibe em tempo real 'Nível de acesso detectado: ADMIN' (linhas 677-688) enquanto o usuário digita, transformando o formulário de cadastro em um oráculo de auto-descoberta do segredo.

Evidência: lib/admin-siape.ts:8-57 (array AUTHORIZED_ADMIN_SIAPE); route.ts:71 'const newRole = siapeNumber ? determineRoleFromSiape(siapeNumber) : "EMPLOYEE"'; route.ts:144 'role: newRole' gravado direto no Prisma; page.tsx:677-688 renderiza o resultado de determineRoleFromSiape(profileData.siapeNumber) client-side.

Impacto: Qualquer pessoa capaz de criar uma conta (qualquer email @ifce.edu.br / @aluno.ifce.edu.br via Google, ou o email pessoal na allowlist) pode inspecionar o bundle JS público ou simplesmente observar o feedback ao vivo da própria tela de cadastro, submeter um dos 47 números durante o onboarding e receber o role ADMIN — acesso total de leitura/escrita a todos os registros de ponto, PII de todos os usuários, gestão de máquinas e segredos de kiosk — sem senha, convite ou qualquer outra credencial.

Recomendação: Mover a decisão de role para fora do alcance do usuário final: (1) nunca aceitar siapeNumber como determinante de role em uma rota que o próprio usuário chama sobre si mesmo; (2) mover a lista de matrículas para uma tabela no banco de dados gerenciada apenas por rotas admin-only, nunca importada por um Client Component; (3) tratar qualquer promoção a ADMIN/SUPERVISOR como uma ação administrativa explícita (POST /api/admin/...) auditada, nunca um efeito colateral automático de completar o próprio perfil; (4) rotacionar/invalidar imediatamente os SIAPEs já hardcoded, já que devem ser considerados publicamente conhecidos.

F2 — PUT /api/users/[id] permite auto-atribuição do campo role (escalação SUPERVISOR → ADMIN)

ALTA

Categoria: Permissões no Frontend vs Backend (relacionado: IDOR)

Local: app/api/users/[id]/route.ts:12-21, 90, 119, 127, 134

Descrição: updateUserSchema (linhas 12-21) aceita role: z.enum(['ADMIN','SUPERVISOR','EMPLOYEE']).optional(). O handler PUT permite edição quando session.user.id === id (linha 90, auto-edição). A linha 119 bloqueia explicitamente a alteração de isActive por quem não é ADMIN, mas não existe bloqueio equivalente para role — updateData espalha validatedData por inteiro (linha 127) e grava via prisma.user.update (linha 134). Como a rota está sob o prefixo /api/users, o middleware (proxy.ts, isAdminApiPath) já exige role ADMIN ou SUPERVISOR para sequer alcançar este handler — um EMPLOYEE não chega aqui — mas um SUPERVISOR (papel que o restante do código trata como estritamente menos privilegiado que ADMIN: só ADMIN pode deletar usuário ou alternar isActive) pode chamar PUT /api/users/<próprio-id> com {"role":"ADMIN"} e se auto-promover.

Evidência: route.ts:90 'const canEdit = session.user.id === id || ["ADMIN","SUPERVISOR"].includes(session.user.role)'; route.ts:119 bloqueia apenas isActive; route.ts:127 'const updateData = { ...validatedData }' inclui role sem filtro; route.ts:134 prisma.user.update grava o role recebido.

Impacto: Escalação de privilégio de SUPERVISOR para ADMIN, contornando a separação de camadas que o resto do código impõe (exclusão de usuário e ativação/desativação são ADMIN-only).

Recomendação: Bloquear explicitamente o campo role no caminho de auto-edição (mesmo padrão já usado para isActive na linha 119): somente um ADMIN pode alterar o role de qualquer usuário, inclusive o próprio. Considerar exigir reautenticação (senha atual) para mudanças de role, e emitir um AuditLog dedicado e alertável para toda alteração de role.

F3 — Relatório "PDF" em /api/reports/download interpola dados de usuário sem escape em HTML bruto (Stored XSS)

ALTA

Categoria: XSS

Local: app/api/reports/download/route.ts:142-273 (função generatePDFReport), especialmente 244-246 e 250-251

Descrição: Diferente do restante da aplicação (JSX, escapado automaticamente), generatePDFReport() monta a resposta por concatenação direta de template string, injetando record.user.name, .email, .role, machine.name e .location dentro de um documento HTML devolvido com Content-Type: text/html (apesar do parâmetro ?format=pdf — nenhum PDF real é gerado, é uma inconsistência funcional adicional). POST /api/users (acessível por ADMIN ou SUPERVISOR) aceita qualquer string com 2+ caracteres para name (app/api/users/route.ts:13, createUserSchema) sem nenhuma filtração de caracteres.

Evidência: route.ts:244 '<td>\${record.user?.name || "N/A"}</td>' sem escape; route.ts:245-246 mesmo padrão para email e role; Content-Type: text/html na resposta (linha ~269).

Impacto: Uma conta SUPERVISOR pode criar um usuário cujo name é um payload HTML/JS; o payload executa no navegador de um ADMIN na próxima vez que esse admin rodar GET /api/reports/download?format=pdf e abrir o arquivo, permitindo roubo de sessão/cookie ou ações adicionais como aquele admin. Cruza a mesma fronteira de confiança SUPERVISOR→ADMIN do Achado F2, por um mecanismo diferente.

Recomendação: Escapar todo dado de usuário antes de interpolar em HTML (ex.: uma função escapeHtml()) mínima, ou migrar esse relatório para JSX/React renderizado server-side, que já escapa automaticamente). Corrigir também a inconsistência de Content-Type: se format=pdf, gerar um PDF de fato (pdfmake, já usado no restante do projeto) em vez de retornar HTML disfarçado.

F4 — GET /api/auth/check-user está desautenticado e devolve PII (incluindo role) de qualquer usuário por email

MÉDIA

Categoria: IDOR**Local:** app/api/auth/check-user/route.ts:6-45

Descrição: O arquivo inteiro não contém nenhuma chamada a getSession — confirmado ao listar todas as rotas de app/api que não chamam essa função (apenas 12 de 65 rotas, todas com motivo legítimo de serem pré-autenticação, exceto esta). A única proteção é um rate limit genérico por IP de 100 req/min (rateLimiters.general). O endpoint aceita um parâmetro email arbitrário e, se existir um usuário correspondente, devolve id, name, role, profileComplete, createdAt e updatedAt.

Evidência: route.ts:6-9 sem getSession; route.ts:13-27 monta a query com o email da query string sem checar sessão; route.ts:29-38 devolve role e demais campos no corpo da resposta.

Impacto: Permite (a) enumeração de usuários contra o formato previsível de email institucional, e (b) reconhecimento direcionado — um atacante pode descobrir especificamente quais emails têm role ADMIN/SUPERVISOR antes de tentar outros ataques (incluindo os Achados F1/F2 acima), tudo sem se autenticar.

Recomendação: Restringir a resposta a { exists: boolean } apenas — a UI de login só precisa saber se deve mostrar 'entrar' ou 'cadastrar', não o nome, role ou timestamps do usuário. Se esses campos forem necessários em algum fluxo legítimo, mover para uma rota autenticada.

F5 — /api/kiosk/recent-activity é público e expõe nomes e localização de todos os registros de ponto do dia

MÉDIA

Categoria: Isolamento**Local:** app/api/kiosk/recent-activity/route.ts:9-48; lib/auth-policy.ts:19

Descrição: O endpoint não tem checagem de autenticação por design (deve alimentar a tela do quiosque físico), mas como /api/kiosk é público em todo o middleware (PUBLIC_PREFIXES) e a rota não aplica nenhuma restrição adicional (nem exige um machineId válido/conhecido), qualquer ponto da internet pode chamá-lo diretamente — não apenas quem está fisicamente no terminal — e, omitindo machineId, obter os 10 eventos de entrada/saída mais recentes do dia em todas as máquinas, incluindo o nome de cada funcionário e o local onde bateu o ponto.

Evidência: route.ts:9 'export async function GET' sem getSession; route.ts:24-26 machineId é opcional e não validado contra a lista de máquinas; route.ts:65-73 devolve record.user.name em texto puro.

Impacto: Exposição contínua e automatizável (scraping) de dados de presença/localização de funcionários — dado sensível sob a LGPD, que a própria docs/SECURITY.md do projeto afirma cumprir.

Recomendação: Exigir machineId válido (rejeitar se não corresponder a uma máquina ativa) e considerar autenticação leve por token de dispositivo (o mesmo x-kiosk-secret já usado em /api/kiosk/init) em vez de acesso totalmente anônimo; aplicar rate limiting dedicado a este endpoint.

F6 — KIOSK_PROVISION_SECRET é um segredo único compartilhado por toda a frota de terminais físicos

MÉDIA

Categoria: Isolamento**Local:** .env.example:38-42; app/api/kiosk/init/route.ts:6-20

Descrição: Todo terminal físico é provisionado com o mesmo valor de KIOSK_PROVISION_SECRET (conforme as instruções de setup em .env.example e a leitura de uma única variável de ambiente em isProvisionSecretValid). A derivação por-máquina do machineSecret (Ponto Forte S1) limita corretamente o raio de explosão desse segredo secundário, mas o segredo de provisionamento em si não é isolado por dispositivo: extraí-lo de um único terminal comprometido/roubado (ex.: do localStorage do navegador daquele quiosque) permite chamar /api/kiosk/init para qualquer machineId do sistema e derivar o machineSecret de todos os terminais.

Evidência: app/api/kiosk/init/route.ts:7 'const provisionSecret = process.env.KIOSK_PROVISION_SECRET' – um único valor para todas as máquinas; .env.example:39-41 instrui configurar 'esse mesmo valor uma única vez em cada dispositivo kiosk físico'.

Impacto: Um único terminal físico roubado/comprometido pode ser usado para forjar QR codes válidos para toda a frota, não apenas para o terminal comprometido — reduzindo o benefício do isolamento por-máquina já implementado.

Recomendação: Emitir um segredo de provisionamento distinto por dispositivo/localização no momento do setup (em vez de um valor global), ou pelo menos suportar rotação independente por máquina para conter um comprometimento sem invalidar a frota inteira.

F7 — CSP mantém 'unsafe-eval'/'wasm-unsafe-eval' justificados por uma dependência que não está mais instalada

BAIXA

Categoria: XSS**Local:** next.config.mjs:8-9

Descrição: O script-src da Content-Security-Policy inclui 'unsafe-eval' e 'wasm-unsafe-eval', justificados por um comentário afirmando serem 'necessário para o WebAssembly do @react-pdf/renderer'. Esse pacote não está presente em package.json, não está instalado em node_modules e não é importado em nenhum arquivo do projeto (confirmado por busca em todo o repositório) — o projeto aparentemente migrou para pdfmake (conforme docs/PDF MIGRATION GUIDE.md) sem apertar a CSP de volta.

Evidência: next.config.mjs:8 comentário citando @react-pdf/renderer; next.config.mjs:9 'unsafe-eval' e 'wasm-unsafe-eval' presentes no array scriptSrc; ausência confirmada do pacote em package.json e node_modules/@react-pdf.

Impacto: 'unsafe-eval' somado ao 'unsafe-inline' já existente reduz substancialmente a capacidade da CSP de conter um payload de XSS caso um seja introduzido futuramente (ex.: um padrão como o do Achado F3 em outro lugar) — a política está fazendo menos do que poderia, sem nenhum ganho funcional atual.

Recomendação: Remover 'unsafe-eval' e 'wasm-unsafe-eval' do script-src e validar o build/testes end-to-end; se alguma dependência futura realmente precisar de eval/WASM, reintroduzir de forma escopada (ex.: um nonce ou hash específico) em vez de uma allowlist ampla e permanente.

5. Recomendações Priorizadas

Ordem sugerida de correção, do maior para o menor risco combinado (severidade × facilidade de exploração):

#	Ação	Achados
1	Corrigir F1 imediatamente: remover a lista de SIAPE-admin de qualquer caminho acessível ao próprio usuário e de qualquer módulo importado por Client Components; tratar toda promoção a ADMIN como ação administrativa auditada.	F1
2	Bloquear alteração do campo role no auto-edit de /api/users/[id] e reauditar todo endpoint que aceite role no corpo da requisição.	F2
3	Escapar (ou reescrever em JSX) a geração de HTML em /api/reports/download; corrigir a inconsistência format=pdf retornando text/html.	F3
4	Reduzir a resposta de /api/auth/check-user a {exists: boolean} e mover qualquer campo adicional para uma rota autenticada.	F4
5	Restringir /api/kiosk/recent-activity a machineId válido e considerar exigir o token de dispositivo já usado em /api/kiosk/init.	F5
6	Migrar de um KIOSK_PROVISION_SECRET global para segredos por-dispositivo, ou suportar rotação independente por máquina.	F6
7	Remover 'unsafe-eval'/'wasm-unsafe-eval' da CSP em next.config.mjs, já que @react-pdf/renderer não está mais em uso.	F7

Anexo — Templates de Issues (Markdown)

Um template pronto para copiar em Novo Issue no GitHub para cada achado de severidade MÉDIA ou superior. Formataado em Markdown (títulos, checklist, bloco de código).

Issue para F1

```
## [F1] Lista de matrículas SIAPE "administradoras" hardcoded e enviada ao bundle do navegador permite auto-promoção a ADMIN

**Labels:** security, severity:critical, security:secrets
**Severidade:** CRITICAL
**Categoria:** Chaves Expostas

### Impacto
Qualquer pessoa capaz de criar uma conta (qualquer email @ifce.edu.br / @aluno.ifce.edu.br via Google, ou o email pessoal na allowlist) pode inspecionar o bundle JS público ou simplesmente observar o feedback ao vivo da própria tela de cadastro, submeter um dos 47 números durante o onboarding e receber o role ADMIN – acesso total de leitura/escrita a todos os registros de ponto, PII de todos os usuários, gestão de máquinas e segredos de kiosk – sem senha, convite ou qualquer outra credencial.

### Evidência
```
lib/admin-siape.ts:8-57 (array AUTHORIZED_ADMIN_SIAPE); route.ts:71 'const newRole = siapeNumber ? determineRoleFromSiape(siapeNumber) : "EMPLOYEE"'; route.ts:144 'role: newRole' gravado direto no Prisma; page.tsx:677-688 renderiza o resultado de determineRoleFromSiape(profileData.siapeNumber) client-side.
```

Arquivos afetados:
- `lib/admin-siape.ts:8-57`
- `app/api/auth/complete-profile/route.ts:71, 144`
- `app/auth/complete-profile/page.tsx:1, 21, 677-688`
- `app/admin/users/new/page.tsx:1, 30, 611-616`

### Recomendação
Mover a decisão de role para fora do alcance do usuário final: (1) nunca aceitar siapeNumber como determinante de role em uma rota que o próprio usuário chama sobre si mesmo; (2) mover a lista de matrículas para uma tabela no banco de dados gerenciada apenas por rotas admin-only, nunca importada por um Client Component; (3) tratar qualquer promoção a ADMIN/SUPERVISOR como uma ação administrativa explícita (POST /api/admin/...) auditada, nunca um efeito colateral automático de completar o próprio perfil; (4) rotacionar/invalidar imediatamente os SIAPEs já hardcoded, já que devem ser considerados publicamente conhecidos.

### Checklist de aceite
- [ ] Correção implementada conforme a recomendação acima
- [ ] Teste automatizado (ou manual documentado) cobrindo o cenário de exploração descrito
- [ ] Revisão de código por outra pessoa da equipe
- [ ] Verificado em ambiente de staging/preview antes do merge
- [ ] AuditLog / log de segurança emitido para esta classe de evento (quando aplicável)
```

Issue para F2

```
## [F2] PUT /api/users/[id] permite auto-atribuição do campo role (escalação SUPERVISOR → ADMIN)

**Labels:** security, severity:high, security:authz
**Severidade:** HIGH
**Categoria:** Permissões no Frontend vs Backend

### Impacto
Escalação de privilégio de SUPERVISOR para ADMIN, contornando a separação de camadas que o resto do código impõe (exclusão de usuário e ativação/desativação são ADMIN-only).
```

Evidência

...

```
route.ts:90 'const canEdit = session.user.id === id ||  
["ADMIN", "SUPERVISOR"].includes(session.user.role)'; route.ts:119 bloqueia apenas isActive;  
route.ts:127 'const updateData = { ...validatedData }' inclui role sem filtro; route.ts:134  
prisma.user.update grava o role recebido.  
...
```

Arquivos afetados:

- `app/api/users/[id]/route.ts:12-21, 90, 119, 127, 134`

Recomendação

Bloquear explicitamente o campo role no caminho de auto-edição (mesmo padrão já usado para isActive na linha 119): somente um ADMIN pode alterar o role de qualquer usuário, inclusive o próprio. Considerar exigir reautenticação (senha atual) para mudanças de role, e emitir um AuditLog dedicado e alertável para toda alteração de role.

Checklist de aceite

- [] Correção implementada conforme a recomendação acima
- [] Teste automatizado (ou manual documentado) cobrindo o cenário de exploração descrito
- [] Revisão de código por outra pessoa da equipe
- [] Verificado em ambiente de staging/preview antes do merge
- [] AuditLog / log de segurança emitido para esta classe de evento (quando aplicável)

Issue para F3

[F3] Relatório "PDF" em /api/reports/download interpola dados de usuário sem escape em HTML bruto (Stored XSS)

****Labels:**** security, severity:high, security:xss
****Severidade:**** HIGH
****Categoria:**** XSS

Impacto

Uma conta SUPERVISOR pode criar um usuário cujo nome é um payload HTML/JS; o payload executa no navegador de um ADMIN na próxima vez que esse admin rodar GET /api/reports/download?format=pdf e abrir o arquivo, permitindo roubo de sessão/cookie ou ações adicionais como aquele admin. Cruza a mesma fronteira de confiança SUPERVISOR→ADMIN do Achado F2, por um mecanismo diferente.

Evidência

...

```
route.ts:244 '<td>${record.user?.name || "N/A"}</td>' sem escape; route.ts:245-246 mesmo padrão para  
email e role; Content-Type: text/html na resposta (linha ~269).  
...
```

Arquivos afetados:

- `app/api/reports/download/route.ts:142-273 (função generatePDFReport), especialmente 244-246 e 250-251`

Recomendação

Escapar todo dado de usuário antes de interpolar em HTML (ex.: uma função escapeHtml() mínima, ou migrar esse relatório para JSX/React renderizado server-side, que já escapa automaticamente). Corrigir também a inconsistência de Content-Type: se format=pdf, gerar um PDF de fato (pdfmake, já usado no restante do projeto) em vez de retornar HTML disfarçado.

Checklist de aceite

- [] Correção implementada conforme a recomendação acima
- [] Teste automatizado (ou manual documentado) cobrindo o cenário de exploração descrito
- [] Revisão de código por outra pessoa da equipe
- [] Verificado em ambiente de staging/preview antes do merge
- [] AuditLog / log de segurança emitido para esta classe de evento (quando aplicável)

Issue para F4

```
## [F4] GET /api/auth/check-user está desautenticado e devolve PII (incluindo role) de qualquer usuário por email

**Labels:** security, severity:medium, security:idor
**Severidade:** MEDIUM
**Categoria:** IDOR

### Impacto
Permite (a) enumeração de usuários contra o formato previsível de email institucional, e (b) reconhecimento direcionado – um atacante pode descobrir especificamente quais emails têm role ADMIN/SUPERVISOR antes de tentar outros ataques (incluindo os Achados F1/F2 acima), tudo sem se autenticar.

### Evidência
...

route.ts:6-9 sem getSession; route.ts:13-27 monta a query com o email da query string sem checar sessão; route.ts:29-38 devolve role e demais campos no corpo da resposta.
...

Arquivos afetados:
- `app/api/auth/check-user/route.ts:6-45`

### Recomendação
Restringir a resposta a { exists: boolean } apenas – a UI de login só precisa saber se deve mostrar 'entrar' ou 'cadastrar', não o nome, role ou timestamps do usuário. Se esses campos forem necessários em algum fluxo legítimo, mover para uma rota autenticada.

### Checklist de aceite
- [ ] Correção implementada conforme a recomendação acima
- [ ] Teste automatizado (ou manual documentado) cobrindo o cenário de exploração descrito
- [ ] Revisão de código por outra pessoa da equipe
- [ ] Verificado em ambiente de staging/preview antes do merge
- [ ] AuditLog / log de segurança emitido para esta classe de evento (quando aplicável)
```

Issue para F5

```
## [F5] /api/kiosk/recent-activity é público e expõe nomes e localização de todos os registros de ponto do dia

**Labels:** security, severity:medium, security:isolation
**Severidade:** MEDIUM
**Categoria:** Isolamento

### Impacto
Exposição contínua e automatizável (scraping) de dados de presença/localização de funcionários – dado sensível sob a LGPD, que a própria docs/SECURITY.md do projeto afirma cumprir.

### Evidência
...

route.ts:9 'export async function GET' sem getSession; route.ts:24-26 machineId é opcional e não validado contra a lista de máquinas; route.ts:65-73 devolve record.user.name em texto puro.
...

Arquivos afetados:
- `app/api/kiosk/recent-activity/route.ts:9-48`
- `lib/auth-policy.ts:19`

### Recomendação
Exigir machineId válido (rejeitar se não corresponder a uma máquina ativa) e considerar autenticação leve por token de dispositivo (o mesmo x-kiosk-secret já usado em /api/kiosk/init) em vez de acesso totalmente anônimo; aplicar rate limiting dedicado a este endpoint.

### Checklist de aceite
- [ ] Correção implementada conforme a recomendação acima
- [ ] Teste automatizado (ou manual documentado) cobrindo o cenário de exploração descrito
- [ ] Revisão de código por outra pessoa da equipe
```

- [] Verificado em ambiente de staging/preview antes do merge
- [] AuditLog / log de segurança emitido para esta classe de evento (quando aplicável)

Issue para F6

[F6] KIOSK_PROVISION_SECRET é um segredo único compartilhado por toda a frota de terminais físicos

****Labels:**** security, severity:medium, security:isolation

****Severidade:**** MEDIUM

****Categoria:**** Isolamento

Impacto

Um único terminal físico roubado/comprometido pode ser usado para forjar QR codes válidos para toda a frota, não apenas para o terminal comprometido – reduzindo o benefício do isolamento por-máquina já implementado.

Evidência

```

app/api/kiosk/init/route.ts:7 'const provisionSecret = process.env.KIOSK\_PROVISION\_SECRET' – um único valor para todas as máquinas; .env.example:39-41 instrui configurar 'esse mesmo valor uma única vez em cada dispositivo kiosk físico'.  
```

Arquivos afetados:

- `.env.example:38-42`
- `app/api/kiosk/init/route.ts:6-20`

Recomendação

Emitir um segredo de provisionamento distinto por dispositivo/localização no momento do setup (em vez de um valor global), ou pelo menos suportar rotação independente por máquina para conter um comprometimento sem invalidar a frota inteira.

Checklist de aceite

- [] Correção implementada conforme a recomendação acima
- [] Teste automatizado (ou manual documentado) cobrindo o cenário de exploração descrito
- [] Revisão de código por outra pessoa da equipe
- [] Verificado em ambiente de staging/preview antes do merge
- [] AuditLog / log de segurança emitido para esta classe de evento (quando aplicável)